The Programming in Java Audit

A Capstone Project for Programming in Java course

Course: Programming in Java

Topic: Library Management

Student Name	Registration Number
Aryam Singh	**24BAI10624**
Slot	**Date of Submission**
E11+E12	**31-03-2026**

1. ABSTRACT

Efficient management of books and records is an essential requirement in libraries, especially when handling large inventories. In many small-scale setups, book tracking is often done manually, leading to issues such as misplaced records, difficulty in tracking issued books, and lack of proper penalty handling for late returns.

The Library Management System developed in this project provides a structured and automated solution to these problems. The system allows users to maintain a digital inventory of books, issue and return books, and track their availability efficiently. It also incorporates a penalty mechanism where overdue returns are charged at a fixed rate per day.

The application is implemented as a console-based Java program and uses core programming concepts such as object-oriented design, collections, file handling, and date-time APIs. The system ensures persistence of data across sessions using file storage.

This project demonstrates how fundamental Java concepts can be effectively applied to build a practical, scalable, and user-oriented solution for real-world.

2. INTRODUCTION

Libraries play a crucial role in academic and professional environments. However, managing a large collection of books manually can be inefficient and error-prone. Traditional methods often lack proper tracking mechanisms for issued books, return deadlines, and penalties.

With the increasing need for digital solutions, even small-scale libraries require systems that can simplify operations without introducing complexity. A lightweight console-based system can significantly improve efficiency while remaining easy to use.

This project introduces a Library Management System built using Java. The system provides functionalities such as book addition, issue and return tracking, availability management, and penalty calculation. It also ensures data persistence, allowing records to be stored and retrieved across sessions.

3. OBJECTIVES

The main objectives of this project are:

1. To design and develop a library management system using Java
2. To maintain a structured inventory of books
3. To implement book issue and return functionality
4. To automate return deadlines (14 days from issue date)
5. To calculate penalties for late returns
6. To implement search functionality for efficient retrieval
7. To ensure data persistence using file handling
8. To apply object-oriented programming concepts

4. SCOPE OF THE PROJECT

The project focuses on creating a simple and efficient library management system for small-scale environments such as educational institutions or personal libraries.

The system includes features such as book inventory management, issue/return tracking, and penalty calculation. It is implemented as a console-based application, making it lightweight and easy to run without requiring additional infrastructure.

The system does not include advanced features such as GUI, database integration, or multi-user access. However, it provides a strong foundation for further enhancements.

5. METHODOLOGY

The development of the Library Management System was carried out using a structured and systematic approach. Each phase of development focused on translating real-world requirements into an efficient software solution using core Java concepts.

5.1 Requirement Analysis

The initial stage involved identifying the key challenges faced in manual library management systems. These included difficulty in tracking issued books, lack of proper return deadlines, absence of penalty calculation, and inefficient handling of large datasets. Based on these observations, the system requirements were defined, focusing on inventory management, issue-return tracking, search functionality, and data persistence.

5.2 System Design

After gathering requirements, the system was designed using object-oriented principles. The design was divided into multiple classes to ensure modularity and maintainability. The Book class was used to represent individual book entities, while the Library class handled shared data such as penalty tracking. The Main class controlled user interaction through a menu-driven interface.

5.3 Implementation

The system was implemented using Java. Core functionalities such as adding books, viewing records, issuing books, returning books, and searching were developed. Special attention was given to maintaining clean logic and ensuring that each operation updates the system accurately.

5.4 File Handling Integration

To ensure that data is not lost between program executions, file handling was integrated into the system. The application uses a text file (books.txt) to store book records. Data is written to the file after each operation and loaded when the program starts, ensuring persistence and continuity.

5.5 Testing

The system was tested under multiple scenarios to ensure correctness and reliability. Tests included handling large datasets, issuing and returning books, calculating penalties for late returns, and validating user input. Edge cases such as empty datasets and invalid inputs were also tested to ensure robustness.

6. SYSTEM DESIGN

6.1 System Overview

The Library Management System operates using a menu-driven interface, where users interact with the system by selecting different options. The program runs continuously in a loop, allowing users to perform multiple operations until they choose to exit.

At the start, the system loads existing data from a file. If no data is available, a default inventory is initialized. Each user action, such as adding or issuing a book, updates the system state and is saved to the file.

6.2 Class Design

- **Book Class**
  The Book class represents the fundamental entity of the system. It stores attributes such as book ID, title, issue status, issue date, and return date. It also includes methods to convert data into string format for storage and retrieval.

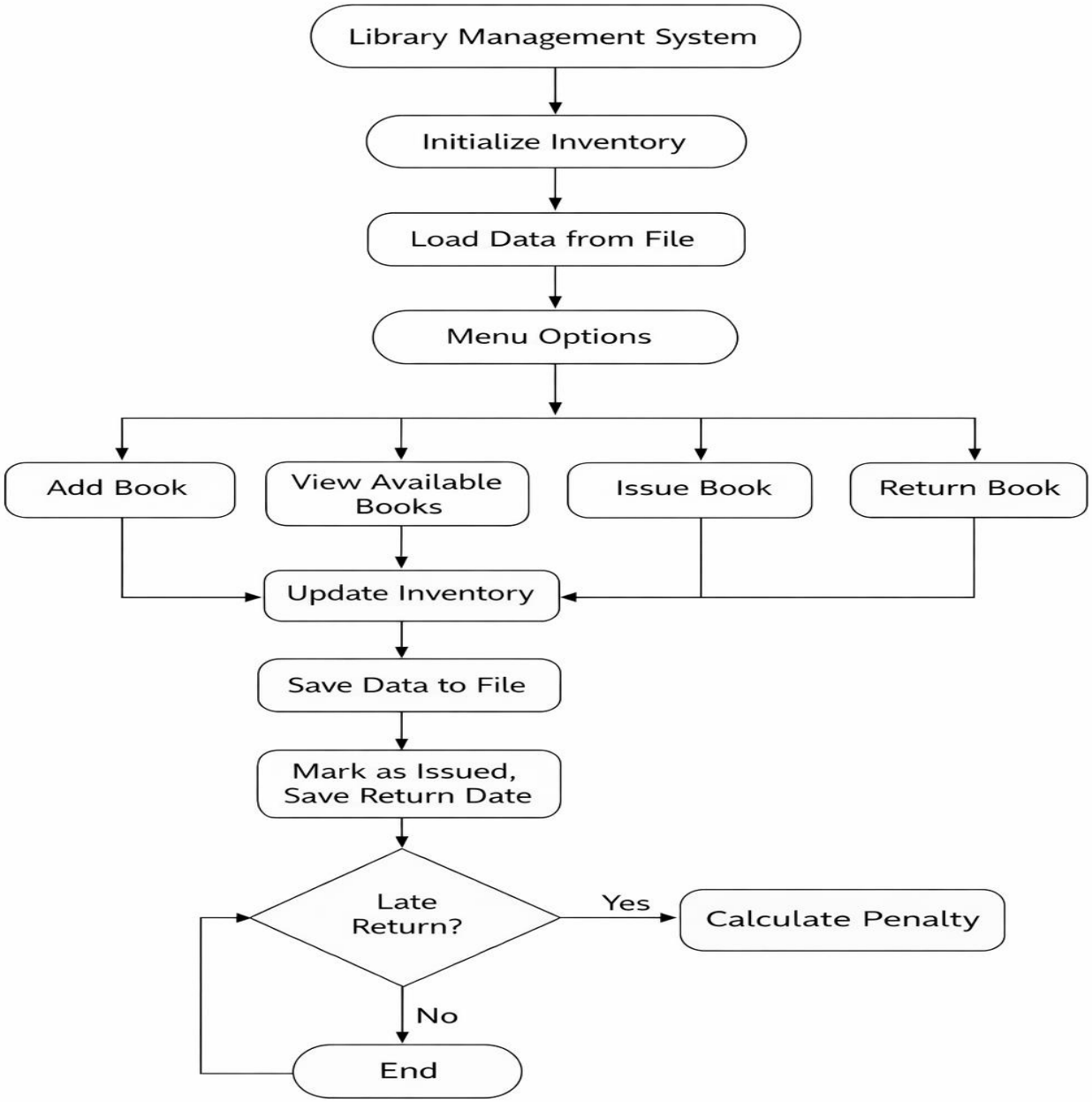
- **Library Class**
  The Library class is responsible for maintaining shared system-level data, such as the total penalty collected. It uses protected access to allow controlled inheritance.

- **Main Class**
  The Main class serves as the entry point of the application. It handles user input using a menu-driven approach and invokes appropriate methods to perform operations.

6.3 Data Structures Used

The system uses the ArrayList data structure from Java's Collections Framework to store book records. ArrayList provides dynamic resizing, allowing the system to handle large numbers of books efficiently. It supports operations such as insertion, deletion, and traversal, making it suitable for this application.

FLOWCHART



The flowchart represents the overall working of the Library Management System in a structured manner. The process begins with the initialization of the system, where the library inventory is either loaded from the existing data file or initialized with default records if no prior data is available.

Once the system is ready, the user is presented with a menu-driven interface that allows selection of different operations. Based on the user's choice, the system performs actions such as adding a new book, viewing available books, issuing a book, or returning a book.

When a book is added, it is inserted into the inventory and the system updates the data accordingly. In the case of viewing books, the system displays either all books or only those that are currently available.

When issuing a book, the system records the user details along with the issue date and automatically assigns a return date after 14 days. The book status is updated to indicate that it has been issued.

When a book is returned, the system checks whether the return is late by comparing the actual return date with the expected return date. If the book is returned after the due date, the system calculates a penalty based on the number of delayed days and updates the total penalty collected.

After every operation, the inventory is updated and the data is saved to a file to ensure persistence. The process continues in a loop, allowing multiple operations until the user chooses to exit the system.

7. IMPLEMENTATION DETAILS

7.1 Book Management

Each book in the system is uniquely identified using an ID and contains details such as title and issue status. The system ensures that duplicate IDs are not allowed, maintaining data integrity.

7.2 Issue and Return System

When a book is issued, the system records the issue date and automatically assigns a return date 14 days later using Java's LocalDate

API. Upon returning a book, the system verifies whether it is overdue and updates its status accordingly.

7.3 Penalty Calculation

If a book is returned after the due date, a penalty is calculated at a rate of ₹10 per day. The system uses ChronoUnit to calculate the number of delayed days and updates the total penalty collected.

7.4 Search Functionality

The system allows users to search for books using keywords. It performs case-insensitive matching on book titles to provide relevant results efficiently.

7.5 Pagination

To handle large datasets (e.g., 1000 books), pagination is implemented. Books are displayed in smaller groups (pages), improving readability and user experience.

7.6 File Handling

The system uses BufferedReader and BufferedWriter for file operations. Data is stored in a structured format in books.txt and is loaded when the application starts. This ensures that all records are preserved across sessions.

8. OUTPUT DESCRIPTION

The system provides a clear and interactive menu-driven interface that allows users to perform various operations. Each operation produces structured output, making it easy to understand the current state of the system.

Users can add new books, view all records, issue books, return books, and search for specific titles. The system also displays statistics such as total books, issued books, available books, and total penalty collected.

The output is designed to be simple, readable, and informative.

MAIN MENU

The below screenshot shows the main menu of the Library Management System. It provides various options such as adding books, viewing records, issuing books, returning books, searching, and displaying statistics. The menu-driven interface allows the user to interact with the system efficiently.

```
===== LIBRARY MENU =====
1. Add Book
2. View All Books
3. View Available Books
4. Issue Book
5. Return Book
6. Show Stats
7. Search Book
8. Exit
```

ADDING A BOOK

The below screenshot shows the main menu of the Library Management System. It provides various options such as adding books, viewing records, issuing books, returning books, searching, and displaying statistics. The menu-driven interface allows the user to interact with the system efficiently.

```
===== LIBRARY MENU =====
1. Add Book
2. View All Books
3. View Available Books
4. Issue Book
5. Return Book
6. Show Stats
7. Search Book
8. Exit
1
Enter Book ID: 1002
Enter Title: Wings of fire
Book added!
```

VIEWING BOOKS

The system displays books in a paginated format to handle large datasets efficiently. Only a limited number of books are shown per page, and the user can navigate using next and previous options. This improves readability and usability.

```
===== LIBRARY MENU =====
1. Add Book
2. View All Books
3. View Available Books
4. Issue Book
5. Return Book
6. Show Stats
7. Search Book
8. Exit
2

--- Page 1 ---
ID: 1 | Physics Vol-54 | Issued: false
ID: 2 | Artificial Intelligence Vol-37 | Issued: false
ID: 3 | Geography Vol-66 | Issued: false
ID: 4 | Geography Vol-8 | Issued: false
ID: 5 | Literature Vol-33 | Issued: false
ID: 6 | Mathematics Vol-37 | Issued: false
ID: 7 | Algorithms Vol-66 | Issued: false
ID: 8 | Literature Vol-6 | Issued: false
ID: 9 | Computer Networks Vol-89 | Issued: false
ID: 10 | Mathematics Vol-57 | Issued: false

1. Next | 2. Prev | 3. Exit
```

ISSUING A BOOK

This screenshot shows the book issuance process. The user enters the book ID, name, and issue date. The system automatically assigns a return date 14 days later and updates the book status to issued.

```
===== LIBRARY MENU =====
1. Add Book
2. View All Books
3. View Available Books
4. Issue Book
5. Return Book
6. Show Stats
7. Search Book
8. Exit
4
Enter Book ID: 202
Enter User Name: Aryam
Enter Issue Date (dd/MM/yyyy): 15/04/2026
Issued! Return by: 29/04/2026
```

9. ADVANTAGES

- The system is simple and easy to use due to its menu-driven interface
- It efficiently handles large datasets using dynamic data structures
- Automatic calculation of return deadlines reduces manual effort
- Penalty calculation ensures accountability for late returns
- File handling ensures data persistence across sessions
- The modular design allows easy extension and maintenance

10. LIMITATIONS

- The system is console-based and lacks a graphical interface
- It does not use a database, limiting scalability for very large datasets
- There is no user authentication or access control
- It is designed for single-user operation and does not support concurrency

11. FUTURE ENHANCEMENTS

The system can be further improved in several ways to make it more robust and scalable. A graphical user interface (GUI) can be developed using Java Swing or JavaFX to improve usability. Database integration can replace file-based storage to handle larger datasets efficiently.

Additional features such as user authentication, role-based access, advanced search filters, and sorting mechanisms can be implemented. The system can also be extended into a web or mobile application for remote accessibility.

12. CHALLENGES FACED

Several challenges were encountered during the development of this project. One of the major challenges was handling file format inconsistencies when reading and writing data, especially after modifying the data structure.

Another challenge was implementing accurate date-based calculations for return deadlines and penalties using Java's LocalDate API. Managing large datasets efficiently required the use of pagination to avoid cluttered output.

Ensuring proper input validation was also critical to prevent runtime errors. Additionally, designing a clean and user-friendly menu-driven interface required careful planning.

These challenges helped in gaining a deeper understanding of Java concepts and improved problem-solving skills.

13. CONCLUSION

The Library Management System developed in this project effectively demonstrates the practical application of core Java programming concepts in addressing a real-world problem. By integrating object-oriented programming principles, file handling mechanisms, and date-time operations, the system provides a structured and efficient solution for managing library records.

The implementation highlights how concepts such as encapsulation, modular class design, and dynamic data structures can be used to build a maintainable and scalable application. The use of Java's LocalDate API ensures accurate handling of issue and return dates, while file-based persistence guarantees that data is retained across multiple sessions. Additionally, features such as search functionality, pagination, and penalty calculation enhance both usability and system efficiency.

The project not only achieves its primary objective of simplifying library operations but also demonstrates the importance of designing systems that are user-oriented and reliable. It provides a clear example of how theoretical programming concepts can be transformed into a functional application with real-world relevance.

Furthermore, the system establishes a strong foundation for future enhancements. It can be extended to include graphical user interfaces, database integration, multi-user support, and advanced features such as authentication and role-based access. These improvements would further increase the scalability and usability of the system.

In conclusion, the project serves as a comprehensive learning experience and successfully showcases the ability to design, implement, and evaluate a practical software solution using Java.